

I've spent a lot of time looking at the details of Tandem's NonStop SQL debit credit benchmark, which they called TopGun, and have come to the conclusion that:

- o They used every trick they could (but that's benchmarking!)
- o The tricks they used are not necessarily applicable in the 'real world'
- o Tandem's Debit Credit is about four times easier than the Digital Debit Credit guidelines I've seen
- o Digital has a marketing challenge in that our conservative Debit Credit should not be directly compared to Tandem's
- o Using the same tricks Tandem did, I believe we can achieve very competitive costs/tps and COO/tps, and that we should do such testing in order to allow the sales force to compare apples to apples for the customer (before moving on to more realistic measures of likely customer-environment performance)
- o The Digital spring product offensive will likely make us much more competitive with Tandem

Without running debit credit on Tandem (yet), we can't accurately assess what a Tandem would do running a conservative debit credit. However, we can look at several of their 'go fast' tricks and make a reasoned assessment of what that trick bought them.

I'd like comments on the following arguments and numbers from those of you familiar with Tandem, TopGun, or Digital debit credit.

Because Tandem 'clustered' multiple systems, I'll use a single 8-processor VLX configuration for these arguments because it scales to standard Digital configurations without getting into lot's of cluster performance arguments. Tandem used four identical 8-processor systems to generate the 208 tps numbers quoted in the press.

Baseline:

8 VLX processors (at 3 mips and 8MB each)
20 disks at 128MB each (they wanted heads) includes 90 day history
tps based on mirrored disks; costs based on mostly unmirrored
1 56KB X.25 line to each processor
each processor had its own set of branch/teller/account files

tps (90%-2 seconds) = 58
5 year COO (discounted maintenance cash flow) \$2,995,000
COO/tps \$51.6K

Source: Tandem publication #84160, TopGun Benchmark Workbook

Restating the Baseline for Conservative Debit Credit

1. Performance at 95%-1 second response time (end-to-end, measured at driver system [which is overly conservative]) drops about 50% based on extrapolation from the data points provided in Tandem's TopGun report. Tandem claims 180-200 milliseconds in the response time is due to the network, driver X.25 process, and driver system dispatching overhead.

I feel very comfortable in downgrading by 50% on this issue.

2. Tandem did NO presentation services, arguing that this was done by an (uncosted) intelligent controller at the branch. This argument has some real world weight, but deviates from the debit credit definition.

Because Tandem's Pathway product uses INTERPRETED COBOL, then ANY presentation services get expensive quickly. The cost of mapping 30 fields per transaction is, I estimate, worth from 20% to 50%. I know our own testing shows that presentation services in TDMS contributes substantially to making debit credit CPU-bound.

3. Tandem, by using the 'intelligent controller at the branch' argument, simulated only 1000 active tellers for 1000 branches, not the 10,000 called for. Doing so eliminated the possibility of branch record contention, and saved memory and virtual circuits. Worth maybe 5%-10%.

4. Every processor had its own comm line to local disk account/branch/teller files (ABT files). 'Not on us' transactions were routed to the proper processor. Tandem gained considerably by partitioning the workload VERY evenly across the available processors, scaling the database at 8 tps/processor. This resulted in an overscaled database (8 processors x 4 systems x 8 tps/processor = 256 tps...they only got 208 tps).

Digital (and any other vendor) could do similar partitioning, so I don't consider this a trick. However, not all (not many!) customer applications lend themselves to the symmetric partitioning Tandem did.

If Tandem had to use central, single ABT databases, then they would hit the wall on disk process throughput or some such software limitation. They would need much less hardware, but they'd get much less work done too.

HPS debit credit testing this spring should shed some light on the cost/value of partitioning.

Let's assume for this memo that partitioning is an accepted debit credit practice. Tandem can use it with no penalty. We can use it too.

5. Tandem gets the same throughput with SQL as with their file system. Sounds competitively bad. In fact, Tandem used proprietary extensions to SQL and accessed the teller and branch files by relative record key. Relative access, of course, is a very fast access method.

Tandem made some claims about doing the account balance update in SQL in the disk process itself, a nice feat. However, NonStop SQL can only do this with single column updates. This means applications that have to update multiple rows will have a longer code path.

Lastly, Tandem treated the ABT database as files. There were no (expensive) JOINS or other SQL constructs used. To Tandem's credit, SQL is not by debit credit. My point is that just because Tandem used NS SQL, we should not immediately counter with Rdb when RMS might do the job. On the other hand, as I conclude below, we may be in better shape than many people think...which is the point of this exercise.

6. Tandem used and costed mirrored disks. Worth maybe 5% to throughput.

Conclusion

A conservative approach to Debit Credit, such as the approach Digital uses for Debit Credit, when applied to Tandem will lead to considerably reduced Tandem throughput. My estimate is 10-15 tps on the baseline described above.

I have no doubt that we can perform today in the 10-15 tps range, if not with one machine, then with a small cluster. New products this year will push us to even higher levels.

Using Tandem's TopGun methodology and definition of debit credit in a Digital debit credit should result in competitive performance and very competitive COO/tps. Partitioning, branch MicroVAXes for presentation services, etc. work just as well in our favor as they did for Tandem. In fact, I think we should put some SMP machines together and generate some 100+ tps numbers if only to say, yeah, we can do that too.

So, Tandem is not as awesomely good as first appears. And Digital is not nearly as bad as corporate mythology would have us believe.

Peter Kastner
Corporate Systems Group
13 January 1988

MEMO

To:

Bill Elliott
Bill Foster
Bill McBride
Bill Thompson
Bob Freiburghouse
Gary Okimoto
Greg Baryza
John Curtis
John Howard
Larry Sherman
Mike Grady
Paul Tucker
Raphael Frommer
Rob Larson
Scott Mitchell

From: Peter Kastner

August 11, 1987

Subject: **Tandem's NonStop SQL Tests -- the TopGun Benchmark**

Attached is a preliminary analysis of the ET-1 benchmark report recently released by Tandem on their massive, 32 VLX processor test of NonStop SQL. The report totals some 580 pages and includes numerous source code listings and detailed benchmark timing results.

The conclusions of Clark Hodder, Raphael Frommer and myself are that a comparable response by Stratus which can pass competitive scrutiny will require five or more man-years of effort by some of Stratus's best people plus the costs of an outside auditor.

We have a number of options in response to Tandem's efforts, including:

- o Using their report as a cookbook to do a similar test
- o Ignoring their tests and continuing with our current version
- o Raising the stakes by doing a 'more compliant' test than they did
- o Deemphasizing ET-1 as a meaningless test of real-world applications (and SQL)
- o Devising our own, new OLTP benchmark standard
- o Embracing another OLTP standard such as RAMP-C (when it is defined)

My conclusion is we will have to do something, some day, to respond to Tandem. Now that we know what Tandem actually did and how they did it, we can begin to assess our response.

Peter Kastner



Working Document on the NonStop SQL Benchmark

Tandem Publication #84160

Author: Raphael Frommer with comments and Observations by Clark Hodder and
Peter Kastner August, 1987

Why did Tandem run this benchmark?

A section of the Workbook is a set of overheads describing the history and results of the so-called Top Gun benchmark. In it, Tandem claims its motives for running the benchmark were to demonstrate that NonStop SQL:

- [] works
- [] is distributed
- [] injects no performance penalty
- [] provides linear growth
- [] is scaleable from EXT to VLX systems
- [] provides good price/performance
- [] has no limit on performance

While no direct or indirect statements about competitors are made, the name of the benchmark, Top Gun, says it all.

General notes:

1. The Account table was partitioned 33 ways (across 32 VLX cpus plus one EXT system), using NonStop SQL's CREATE TABLE syntax. The Branch and Teller files were partitioned in a similar way. Unlike NonStop SQL, ENSCRIBE does present a practical limit to the maximum number of partitions (16), so the ENSCRIBE database and ENSCRIBE benchmark was limited to 16 VLX processors.

2. The auditor (Tom Sawyer, from Codd & Date Consulting Group) confirmed linear expandability, but only within 10% for the largest configuration:

	16-cpu	32-cpu
Predicted (from 8-cpu run)	116	232
Actual	106	208
Divergence	8%	10%
Inter-node xactions:	7.5%	11%

Note: The inter-node transactions were those inter-branch transactions (the specification demands that these constitute 15% of the activity) that wound up traversing a fiber optic or 9.6KB EXPAND link.

3. In the included paper by Anon Et Al (A Benchmark of Tandem's NonStop SQL Demonstrating Over 200 Transactions Per Second), on page 20 there is the admission that the increased cost of network-distributed transactions causes growth to be .9x linear rather than 1x.

This is interesting in light of the fact that IBM gives the System/88 credit for being linear, but only at some fraction of unity. The last time I heard a number, it also was 90%. So now we have Tandem agreeing that it's OK to be linear at a slope other than unity!

4. Each VLX cpu got its own 56KB line; the EXT-10 2-cpu system got one line. Four bit sync controllers per 8-cpu VLX system were used, so each controller handled two lines. In the old days, this would imply the two io processes would run in a single cpu. But nowadays it may be possible to split ownership of a controller, thus smoothing the comm load out across the 8 cpus.

What makes this more interesting still is the Mackie diagram provided, which seems to show two bit sync controllers (4 lines) strapped between cpus 2 and 3, and two bit sync controllers (4 lines) strapped between cpus 6 and 7. This seems to prove conclusively that maximally 4 cpus got directly involved in X.25 communications. At a minimum, it would seem that a VLX cpu handled two 56KB X.25 lines.

The Mackie diagram implies no spare bitsync comm controllers. In the event of a comm failure, two lines (160 branches/1600 tellers) are lost.

5. Although each VLX cpu had to handle 80 branches (800 tellers total), the transactions came in on one X.25 line. Each VLX got 2.5 PATHWAY TCPs, on average, to accomplish this. TCP code and data came to about .5MB/VLX cpu. Twenty NonStop SQL servers on a VLX consumed about 0.5MB.

6. Tandem originally estimated a VLX could do 8TPS, so they scaled their database accordingly. A mirrored disk per VLX cpu stored:

80 branches	8KB
800 tellers	80KB
800,000 accounts	80MB

And they fit these on Tandem's smallest (V8-class) disk. A 1.6 MB disk cache per mirrored disk partition was sufficient to keep all branches, tellers and the account index pages resident in main memory.

Tandem claims to have oversized the data base to handle 256 tps.

7. TMF did group commits. On average, one audit flush was done per 5 commits. Because the audit trail was duplexed, this amounts to .4 physical writes to the audit trail per ET-1.

8. SQL's higher performance is attributed to slightly fewer cpu instructions than ENSCRIBE and reduced message flow. In updating the account table, NonStop SQL "sends half as many messages and one quarter the number of message bytes".

9. A graph of cpu busy/transaction seems to confirm Tandem's claim that NS SQL is less costly than ENSCRIBE: Discprocess busy/ET-1 was 24.5 ms in the NS SQL version versus 28.7 ms in the ENSCRIBE version. Curiously, the difference, 4.2 ms, is eaten up by the reduced efficiency of the COBOL server (the ENSCRIBE COBOL server consumed 4.5 ms less than the COBOL 85 SQL server). Most interesting is the fact that the NonStop SQL implementation, per the suggestion of the auditor, was augmented to check for overdrafts. The ENSCRIBE version, coded in late 1986 by Praful Shah, apparently did not.

10. The history file was composed of 4KB blocks, and required two I/Os for every 80 transactions. At 64TPS this is about 1.25 I/Os per second.

11. Page 6.8 gives us the 90-day history file requirement. Page 4.4 defines 1 tps of history at 2.59MB based on 24 hour/7 day operations at 1/3rd of peak rate. Sounds good enough to emulate their definition.

12. Page 6.7 in the Top Gun Sizing section includes the very significant statement: "The autorestart option will be for TCPs, so no backup TCP or checkpointing to the backup TCP will be required. Additionally IDS support will be used for the terminals." This means that the implementation was not as fault tolerant as Tandem could have made it. In fact, a processor failure would cause a new TCP to be started on an alternate CPU, and 'terminals' would get a login message. Further, tellers would not know what transactions had posted or not, so inquiries would be needed to see what had to be reentered or not. Use of autorestart and IDS reinforce our claim that PATHWAY TCPs and SCOBOL are expensive and that, for best performance, their involvement must be limited to the bare minimum. Our attention should be focussed on how to take advantage of Tandem's lack of requester code and fault tolerance.

On the same page, there is an inference that one VLX system has no difficulty supporting 640 concurrent X.25 circuits.

13. Pricing:

5 year life cycle VLX costs in the \$43K/TPS price range is only possible if database disks are unmirrored and the 90 day history file is omitted. Their "compliance plus" prices jump to \$56K/TPS, and provide dual data paths to 90 days of (assumed duplexed) history, duplexed database and Audittrail disks.

Computing Net Present Value on maintenance cash flows at a 15% discount rate clearly helps their life cycle costs.

14. Tandem used inter-arrival rates as a means of governing the exponential distribution of transactions. They implemented it in a way that forced reality from theory, because new transactions were not started until old ones came back to the driver and it was discovered that it was already late to dispatch another transaction. Hodder believes (and Kastner agrees) that using an exponential distribution of teller think times is not only more 'real world', but free of discrepancy between theory and reality.

15. The simulator reports clearly show that a 15 minute measurement window under steady state conditions was chosen from a 40 minute test period. Thus, it's OK to throw out the costs of test startup.

Things that made life easier for Tandem:

1. "Terminals" were driven in record mode using PATHWAY's Intelligent Device Support. Presentation services were assumed to be done in the terminal, which Tandem assumes would be ATMs in any modern day implementation. Clearly, Tandem made the leanest possible server in order to minimize the amount of interpreted SCOBOL code.

2. A concentrator at each branch was assumed to handle the 10 "terminals" there. This had the effect of reducing the number of sessions by a factor of 10 for the same transaction rate when compared to the standard. This is arguably 'real world', but we could use a circuit per teller if we wanted to. Our choice.

3. Response time was relaxed to 90%-ile 2 second response time (but recall that response time was measured at the driver system, thus adding in its X.25 overhead as well).

Tandem failed to live up to its promise of disclosing 95%-ile 1 second response times (see page 6.11), the ET-1 'standard'. Analysis of Tandem's published response times and throughput from 143 tps to 229 tps shows 95%-ile-1 second response times ranging from 1.3 to 5.1 seconds. Extrapolation from these data show a 32 processor system running about 100 tps at 95%-ile-1 second (measured the very conservative but practical way Tandem measured response times). We should carefully examine the opportunities to do our tests at 95%-1 second and hold Tandem to the same standard, or see what advantages we get going to 90%-2 seconds.

4. Teller and branch updates employed a WHERE clause that used the SYSKEY feature of NonStop SQL. "This field is not a data value in the row; it is the relative record number - in effect a direct reference." This departure from ANSI Standard SQL was cited by the auditor as "removing the access transparency normally gained by SQL". It also keeps the code from being portable.

5. The auditor stated that "the history database did not have physical disks allocated to cover the ninety day requirement. This is a minor matter since this database is updated at the end. The priced configuration should have the correct amount of disk included." Indeed Tandem wound up furnishing several sets of prices, with and without history disks.

6. The configurations are not fault tolerant (comm) and turning off TCP checkpointing saved them a lot of overhead. Tandem declined the auditor's suggestion (page 5.1) that various hardware failures be tested during the measurement cycle. We definitely should maintain our moral 'high ground' by running tests with fully duplexed hardware and assessing failures under load. This is one of our few uniquenesses.

7. Page 4.24 says 'NonStop SQL subcontracts single variable queries to remote servers'. This implies that multi-variable and (surely) multi-file queries are more expensive than the minimal efforts Tandem took.

Things that made life harder for Tandem:

1. SQL had the discprocess check that debits did not cause account balances to become negative. In a non-SQL environment, this is a trivial test. One Stratus avenue of approach is to add code to the server to turn the transaction into a more meaningful Demand Deposit Accounting application. We could develop code which uses all three files (which would require much more expensive SQL 'joins' than the approach Tandem took) with code that does something like:

```
IF ACCOUNT-TYPE = COMMERCIAL THEN
    ADD TRANSACTION-AMOUNT TO TELLER-COMMERCIAL-TOTALS
    ADD TRANSACTION-AMOUNT TO BRANCH-COMMERCIAL-TOTALS
ELSE
    ADD TRANSACTION-AMOUNT TO TELLER-PERSONAL-TOTALS
    ADD TRANSACTION-AMOUNT TO BRANCH-PERSONAL-TOTALS.
```

We have to decide whether SQL is a benefit to DebitCredit or whether the abilities of Stratus SQL (when announced) is best shown on VOS file system files.

2. The system was measured over ten minute periods and the average for each period was used to compute the response time curves and consequent TPS ratings. However, analysis of the driver code indicates they started the test by flooding the system with transactions. Perhaps a better way is to start off with random arrivals, which builds more slowly but avoids the deep pit that's dug when the system is initially flooded.

3. Response times included X.25 overhead within the driver system. Tandem estimates this added 150 to 180 ms to the reported response time. On the whole, this is OK at 2 second response times but very costly if we are using 95%ile-1 second timing. One alternative is to timestamp transactions in the requester just before they are handed off to X.25, and stamping again when the driver application gets the transaction. Dividing the difference in times in half gets us the average of one X.25 cost plus half the line transmission overhead. That is still more rigorous than the ET-1 'standard' which measures last bit in to first bit out of the driven module.

4. All disks, not just the logs, were duplexed. In the benchmark, only one disc was configured for the History table, but in pricing the system the 90-day history file of each node was sized at 6.7GB [There is a claim that this would fit on 14 mirrored volumes, but my calculations show these 13.4 physical gigabytes would require 16 volumes, or 2 XL-8 disk subsystems.]

5. The message interarrival rate was not uniform, but instead exponential, with an average of 10 seconds per branch. The auditor admitted this technique is the most difficult for OLTP system to accommodate. See comment 14 above.

Puzzlements

1. In several places within the workbook, the EXT-10 2-cpu system was claimed to be less powerful than a single VLX cpu. But Tandem's publicly released numbers for both processor types contradict this. An EXT-10 ought to be capable of 4 ET-1/sec, while a single VLX cpu should do 6.25. 4 is certainly not less than half of 6.25.

2. Why were 8-cpu systems, not 16-cpu systems configured? Is this confirmation of a (counter-intuitive) rumor that it's better to FOX-link two 8-cpu systems versus using one 16-cpu VLX?

3. Tandem used V-8 (128MB) disks for the tests, then factored in a huge number of XL8 (400MB) disks to hold the history. Was this just sloppy summarization? They might have been able to reconfigure the disks to partition the active files over a bunch of XL8's that also contained lots of inactive history storage. When we lay out our benchmark we should carefully decide on the best mix of disk size vs number of disks. Also, it wouldn't hurt to actually have the history file(s) exist...Tandem played a little loose here.

4. Page 6.1 and subsequent pages in the benchmark sizing section are headed 'TG .3KTPS Benchmark'. TG means Top Gun. '.3KTPS' means 300 tps to Kastner. Did Tandem originally expect to get 300 tps from this system? Based on the 1986 VLX announcement at 40 tps for 4 cpu's, I believe $30 * 10 = 300$ tps was the expected throughput level. This means Tandem is now working on any bottlenecks they discovered and we can expect them to raise the performance levels over time with software improvements.

Hodder's Observations on the Topgun Benchmark Report (with comments by Kastner)

Tandem's Topgun benchmark demonstrated over 200 ET1 transactions per second. This report is very significant in several ways:

*** Resources**

Tandem devoted a considerable set of resources to this benchmark. The testbed consisted of \$8 Million worth of hardware dedicated for almost two months. The total project spanned at least

seven months. The personnel involved were most of their very top performance, data base, and competitive people. I would estimate that they spent at least 5 man years on this project. Some of the specific people involved were:

Frank Clugage Cupertino DBMS Product Marketing Manager
Jim Gray Cupertino DBMS Guru, Technical Lead ?
Jim Enright Cupertino Performance Expert
Nhan Chu Cupertino Queueing Theory Expert (Wrote Driver)
Pratul Shah Cupertino Stratus Competitive Expert
 Wrote Enscribe server code
Harald Sammer Frankfurt High Performance Research Center Mgr.
Gerhard Huff Frankfurt HPRC
TLW (?) Frankfurt HPRC (Wrote B'mark code)
Tom Sawyer Codd & Date Auditor

** Full Disclosure*

Tandem has established a level of disclosure that must now be respected by any vendor in the future. They have completely redefined the playing field. The report includes:

Auditor's Report
Benchmark Source Code
Driver Source Code
Database Layout
Pricing Information for Cost/Tx
Detailed Response Time Histograms

It will be difficult for other vendors to claim ET1 numbers without similar backup materials. Full disclosure also implies a much more precise compliance with the ET1 standard itself, with any deviations explicitly noted. It means that a substantial amount of effort must be spent cleaning up code and formatting results. An auditor will probably be required. Tom Sawyer appears to have done quite a lot of work in the Topgun report. There are numerous changes in the benchmark code and in the driver per his requests.

** New ET1 Definition*

Topgun establishes a new defacto ET1 standard. They have made some changes to the original ET1 definition, and have defined some areas that were not well defined before. Specifically:

A. Transaction Protection is required. They have not released any unprotected numbers at all. I do not see any good way that we can put unprotected numbers up against their protected numbers. Any prospect exposed to the Tandem Topgun slides will be expecting TP protected numbers. We can't really argue with this. The original ET1 strongly implied TP was required, but Tandem left us a window by announcing old ET1 numbers with and without.

B. The 90 Day History File is now defined to be 2,590,000 records per TPS. (An 8 hour day would be $3600 * 8 * 90 = 2,592,000$.) We may as well use this number. It is good to have an agreed upon size.

C. The number of active tellers is reduced to one per branch, instead of ten per branch. This is a nontrivial change. It means that there is virtually no possibility of record lock contention any more. (The old benchmark had ten tellers contending for one branch record. This gave

us problems under our old TP scheme.) It greatly reduces the memory requirements of the benchmark, there are 1/10 as many virtual circuits, requestor threads, etc. It does greatly simplify setting up a large benchmark. Since our memory is not cheap, I would go along with this change as well.

D. The transaction arrival pattern should be random. This is a good change in the direction of realism. The original ET1 merely specified that the transactions came from each teller at 100 second intervals. This gave unrealistically low response times. Topgun attempted to randomize the arrival time between transactions. This is not quite satisfactory since some interarrival times will be less than the response time. (e.g. a particular transaction completes in say, three seconds. I roll my random dice and decide to send the next transaction two seconds after the last one started. This is impossible, since it implies that I start the next transaction one second ago.) In these cases Topgun simply sends the next transaction immediately. This is not really correct. If they truly want an exponentially distributed interarrival time, they should send the next transaction immediately, and give it a starting timestamp of one second earlier. That one second should count against the response time, and should be used in the following calculation for dispatching the next transaction. If they had done this correctly, their response times would have been even worse than they were. And they were pretty bad. (See below). Oddly enough, it appears that Tandem never did figure out what was wrong with their driver. They have graphs plotting the theoretical arrival pattern vs the observed pattern, and they don't match.

They also noted that they were not getting the expected throughput. (i.e. if the driver tried to average 200 tps, they would observe 180 tps.) Either they didn't figure it out, or they were cheating a little to save their response time.

I would not do the driver this way. I would randomize the think time between transactions. This is much cleaner, and more realistic. If we want to average 10 seconds between transactions, and our average response time is one second, we simply set the average think time to nine seconds.

E. Response times were measured from the driver system. The original ET1 specified response times from the last bit in the x.25 line to the first bit out. It is much easier to measure from the driver, so it seems like a good change, even if it adds .2 seconds to response times.

F. They relaxed the original throughput definition. Old ET1 specified that the throughput be measured when 95% of the transactions finished in less than one second. Tandem has relaxed that requirement to be 90% of the transactions finishing in two seconds. This is a substantial change that they required because of the adverse queueing nature of their system. (See the SESC article I wrote on G200_Sizing.) To get the high throughput numbers Tandem ran the benchmarks with the CPUs 80% to 90% busy. This results in severe queueing and response time problems. Before we throw many stones, however, we need to see how well we do ourselves with TP and random arrival times.

G. The terminals have been redefined. Instead of 3270 style blockmode terminals they are now intelligent cluster controllers sending a formatted x.25 message. This saves a nontrivial amount of CPU time per transaction. In fact, Tandem had absolutely no code for formatting an output buffer. This does simplify the benchmark some more.

H. Multiple History files seem to be OK. In very high volume applications, writing to a single sequential file always becomes the ultimate bottleneck. From a talk by Jim Gray last year I heard Tandem was wrestling with this problem. Apparently they punted, and took the easy way out. This should undermine any criticisms they have about our multifile implementation.

I. All disks were mirrored. This makes sense for us and Tandem. I doubt any other vendors will worry about it. The transaction logs are required to be mirrored.

J. The benchmark was audited. I imagine it is likely that prospects will start to expect this of any other vendor claiming ET1 numbers.

Tandem's GoFast Tricks

Pathway Hacks.

The TCP's did almost nothing. No screen formatting (they did not even include a Screen Section) as the terminal type was 'intelligent'. Almost no data movement at all, the x25 message is sent directly to the server without modification or movement. The reply is moved whole to the x25 output buffer. The TCPs were NOT run NonStop. They did no checkpointing. Theoretically they are relying on the autorestart facility. The transaction loop consists of the following (pseudo) SCOBOL:

```
Loop.  
  Begin-transaction.  
  Send tx-data to "eta"  
    Reply-code yields etc.  
  If <ok>  
    End-transaction  
    Move server-data to x25-out  
    Send message x25-out  
    Reply yields tx-data  
  Else <error>
```

TMF Hacks.

The delta cost of a TMF transaction is actually proportional to the number of disk processes touched by a transaction. This is because each disk process that participated in a TMF transaction has to send a message to the Audit Process (the disk process that owns the Audit Trails). The Topgun team carefully partitioned the data files so that all the records corresponding to a particular branch were on the same physical disk. They call that the ATB disk (Account-Teller-Branch). This reduces the number of audit messages from three to one. The disk procs also buffer the writes to the audit procs, the net result is that the ATB disk process only sends .4 messages to the audit disk procs per transaction, and does .5 checkpoints per transaction. The History file is common for each node, on a disk by itself. The buffering works even better for this disk, it sends .05 messages to the Audit procs per transaction, and does an equal number of checkpoints. The Audit disk process winds up receiving .5 msgs per tx. For each message it does one physical write and one checkpoint to its backup process. The TMF Monitor process receives 1.3 messages per transaction.

File Hacks.

Tandem set up their files so that almost all logical IOs are handled by cache hits. Teller, Branch, and History are all in cache. The only physical IOs are one physical read of the Account file, and one physical mirrored write of the account file. We need two physical reads of the affile, *Account File* and one physical mirrored write of the account file. We need two physical reads of the account file per transaction, due to our indexed file type (vs their VSAM style files.)

The above file is also available in Microsoft WORD format.